JavaScript



Язык, асинхронность и event loop

Frontend-разработчик

Что будем уметь

- Читать и писать небольшой JavaScript-код
- Работать с функциями, объектами и массивами
- Обрабатывать действия пользователя и загружать данные
- Объяснять порядок выполнения асинхронного кода

Сквозной пример: список задач

Три уровня из прошлой темы

Уровень	Примеры
JavaScript	Значения, функции, объекты, Promise
Движок	Выполнение кода и управление памятью
Браузер	DOM, события, сеть, таймеры и отрисовка

У браузерных API есть свои правила выполнения

Первая программа в браузере

```
<!-- index.html -->
<script type="module" src="./main.js"></script>

// main.js
console.log("Привет, JavaScript!");
```

HTML подключает файл.

Результат виден в Console.

Открываем страницу через локальный HTTP-сервер.

Модуль автоматически использует строгий режим

Как читать небольшой фрагмент

```
const title = "Изучить JavaScript";
let completed = false;

// Отметим задачу выполненной
completed = true;
console.log(title, completed);
```

const и let объявляют переменные.

= присваивает значение.

Вызов функции использует круглые скобки.

JavaScript различает регистр: title и Title различны

const и let

```
const course = "Frontend";
let count = 1;

count = count + 1;
console.log(course, count);

// course = "Другой"; // TypeError
```

const запрещает новое присваивание переменной.

let допускает его.

var встретится в старом коде.

В примерах используем const, а при переприсваивании — let

Значения и типы

Тип	Примеры
number, bigint	42, 3.14, 42n
string, boolean	"Задача", true
undefined, null	Нет значения / явное отсутствие
symbol	Уникальный идентификатор
object	Объект, массив, функция

Семь примитивных типов и объекты

Динамическая типизация

```
let value = 12;
console.log(typeof value);

value = "двенадцать";
console.log(typeof value);

console.log(typeof null);
```

Вывод:
number
string
object

Последняя строка — историческая особенность typeof.

Тип есть у значения. Переменная может получить другое значение

undefined и null

```
let selected;
const task = { title: "Git" };

console.log(selected);
console.log(task.deadline);

selected = null;
console.log(selected);
```

undefined появляется, когда значение не задано.

null часто задают явно: «сейчас ничего не выбрано».

Смысл отсутствующего значения закрепляет код приложения

Числа и преобразование ввода

```
const input = "15";
const count = Number(input);
console.log(count + 1);

console.log(Number.isNaN(Number("кот")));
console.log(0.1 + 0.2 === 0.3);
```

Вывод:
16
true
false

number использует двоичное представление с плавающей точкой.

Преобразование ввода и проверка результата — отдельные шаги

Строки и шаблоны

```
const title = "  Изучить JS  ";
const count = 3;

console.log(title.trim());
console.log(`Осталось задач: ${count}`);
console.log("JavaScript".includes("Script"));
```

Обратные кавычки создают шаблонную строку.

${...} подставляет результат выражения.

Методы строки возвращают результат, исходная строка не меняется

Неявные преобразования

```
console.log("5" + 1);
console.log("5" - 1);
console.log(Number("5") + 1);

console.log(Boolean("false"));
```

Вывод:
"51"
4
6
true

Непустая строка "false" истинна в условии.

На границах приложения полезны явные преобразования

Сравнение значений

```
console.log(5 == "5");
console.log(5 === "5");
console.log(5 !== "5");

const a = { id: 1 };
const b = { id: 1 };
console.log(a === b);
```

Вывод:
true
false
true
false

Разные объекты сравниваются по идентичности.

Для обычных сравнений в курсе используем === и !==

Условия

```
const count = 2;

if (count === 0) {
  console.log("Задач нет");
} else if (count < 5) {
  console.log("Задач немного");
} else {
  console.log("Много работы");
}
```

Условия проверяются по порядку.

Выполнится первая подходящая ветка.

=== сравнивает, = присваивает.

Фигурные скобки показывают границы блока

Что считается ложным в условии

Falsy	Truthy: частые неожиданности
false	"false"
0, -0, 0n	"0"
""	" "
null, undefined, NaN	[], {}

Пустой массив и пустой объект считаются истинными

Логические операторы

```
const title = "JS";
const done = false;

console.log(title !== "" && !done);
console.log("" || "Без названия");
console.log(0 || 10);
console.log(0 ?? 10);
```

Вывод:
true
Без названия
10
0

?? подставляет значение только для null и undefined.

&& и || вычисляют операнды по необходимости

Безопасный доступ к вложенным данным

```
const task = {
  title: "JS",
  assignee: null
};

const name = task.assignee?.name ?? "Никто";
console.log(name);
```

?. прекращает цепочку, если слева null или undefined.

?? задаёт значение по умолчанию.

Отсутствие данных должно иметь понятный смысл

Цикл for

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}
```

1. let i = 0 выполняется один раз

2. i < 3 проверяется перед итерацией

3. i++ выполняется после тела

Вывод: 0, 1, 2

Перебор значений: for...of

```
const titles = ["Git", "HTML", "JS"];

for (const title of titles) {
  console.log(title);
}
```

Массив содержит несколько значений.

На каждой итерации title получает следующее значение.

for...in перебирает ключи свойств. Для значений массива нужен for...of

while, break и continue

```
let attempt = 0;
while (attempt < 5) {
  attempt++;
  if (attempt === 2) continue;
  if (attempt === 4) break;
  console.log(attempt);
}
```

continue пропускает остаток текущей итерации.

break завершает цикл.

Вывод: 1, 3.

Условие завершения должно становиться достижимым

Функция: вход, работа, результат

```
function total(price, count) {
  return price * count;
}

const result = total(120, 3);
console.log(result);
```

price и count — параметры.

120 и 3 — аргументы вызова.

return отдаёт результат вызывающему коду.

Определение функции и её вызов — разные действия

Возврат и значение по умолчанию

```
function greet(name = "Гость") {
  return "Привет, " + name;
}

function logTitle(title) {
  console.log(title);
}

console.log(greet());
console.log(logTitle("JS"));
```

Вывод:
Привет, Гость
JS
undefined

Без return функция возвращает undefined.

return завершает текущий вызов функции

Функция как значение

```
function twice(value) {
  return value * 2;
}

function apply(value, operation) {
  return operation(value);
}

const transform = twice;
console.log(apply(7, transform));
```

Функцию можно записать в переменную, передать аргументом и вернуть.

Так работают функции первого класса.

twice передаёт функцию, twice(7) вызывает её

Стрелочные функции

```
const double = value => value * 2;

const normalize = title => {
  return title.trim();
};

const makeTask = title => ({ title, done: false });

console.log(double(4));
console.log(makeTask("JS").done);
```

Без фигурных скобок результат выражения возвращается сразу.

Для возврата объекта нужны скобки вокруг литерала.

У стрелочной функции также особые правила this

Область видимости

```
const title = "Снаружи";

if (true) {
  const title = "Внутри";
  console.log(title);
}

console.log(title);
```

Вывод:
Внутри
Снаружи

let и const видны внутри своего блока и вложенных блоков.

Вложенное объявление может скрыть внешнее имя

Объявление и момент доступа

```
sayHello();
function sayHello() {
  console.log("Привет");
}

// console.log(count); // ReferenceError
const count = 3;
```

Объявление function доступно раньше своей строки в этой области.

let и const нельзя читать до инициализации.

Практическое правило: переменные объявляем до использования

Замыкание

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}

const next = makeCounter();
console.log(next());
console.log(next());
```

Функция сохраняет доступ к окружению, где была создана.

count остаётся доступен после завершения makeCounter.

Вывод: 1, 2. У нового makeCounter() будет свой счётчик

Объект и свойства

```
const task = {
  id: 1,
  title: "Изучить JS",
  done: false
};

task.done = true;
console.log(task.title);
console.log(task["done"]);
```

Объект объединяет связанные данные.

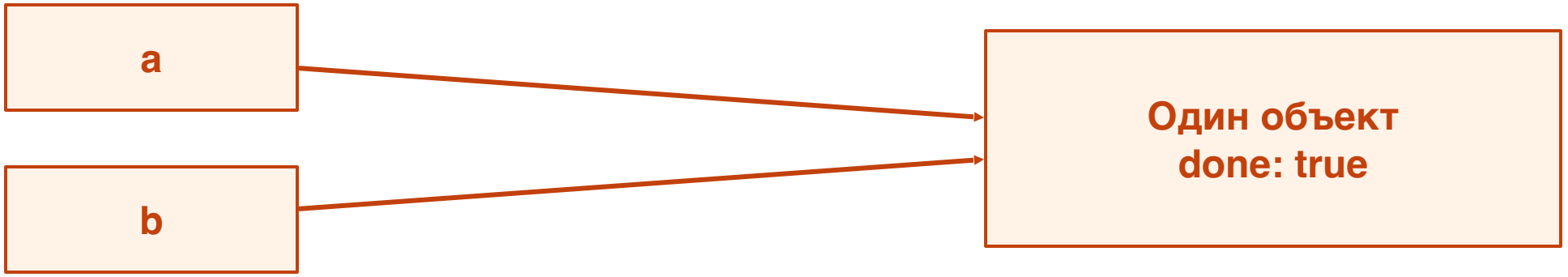
Точка обращается к известному имени.

Квадратные скобки допускают вычисляемый ключ.

Для объекта не обязательно сначала объявлять класс

Две переменные и один объект

```
const a = { done: false };
const b = a;
b.done = true;
```



a.done тоже станет true: a и b указывают на один объект

const и изменение объекта

```
const task = { title: "JS", done: false };

task.done = true;
console.log(task.done);

// task = { title: "CSS" }; // TypeError
```

Изменение свойства разрешено.

Новое присваивание переменной запрещено.

const фиксирует связь переменной со значением

Копирование объекта: один уровень

```
const original = {
  title: "JS",
  owner: { name: "Аня" }
};
const copy = { ...original, title: "CSS" };
copy.owner.name = "Ира";

console.log(original.title);
console.log(original.owner.name);
```

Вывод:
JS
Ира

Вложенный owner по-прежнему общий.

Оператор ... создаёт поверхностную копию

Массивы

```
const titles = ["Git", "HTML"];
titles.push("JS");

console.log(titles[0]);
console.log(titles.length);
console.log(titles[99]);

const last = titles.pop();
console.log(last);
```

Индексы начинаются с нуля.

push и pop меняют массив.

Несуществующий элемент даёт undefined.

Массив — объект с операциями для последовательности значений

filter и map

```
const tasks = [
  { title: "Git", done: true },
  { title: "JS", done: false }
];

const titles = tasks
  .filter(task => !task.done)
  .map(task => task.title);

console.log(titles.join(", "));
```

filter оставляет подходящие элементы.

map преобразует каждый элемент.

Оба возвращают новый массив.

Элементы-объекты при этом не копируются автоматически

Поиск и проверка массива

```
const tasks = [
  { id: 1, done: true },
  { id: 2, done: false }
];

console.log(tasks.find(t => t.id === 2).done);
console.log(tasks.some(t => !t.done));
console.log(tasks.every(t => t.done));
```

find возвращает элемент или undefined.

some проверяет, есть ли подходящий элемент.

every проверяет все элементы.

Для неизвестного результата find сначала проверяем наличие

Сортировка и изменение данных

```
const scores = [2, 10, 1];

const sorted = [...scores].sort((a, b) => a - b);

console.log(sorted.join(", "));
console.log(scores.join(", "));
```

Вывод:
1, 2, 10
2, 10, 1

sort меняет массив. По умолчанию сравнивает как строки.

Здесь сначала создаём копию, затем сортируем числа

Деструктуризация и rest

```
const task = { id: 7, title: "JS" };
const { id, title } = task;

const [first, ...others] = [10, 20, 30];

console.log(id, title);
console.log(first, others.length);
```

Деструктуризация извлекает значения.

...others собирает оставшиеся элементы.

Похожий синтаксис ... может означать rest или spread.

Деструктуризация не создаёт глубокую копию

Метод и this

```
const task = {
  title: "JS",
  print() { console.log(this.title); }
};

task.print();
const detached = task.print;
// detached(); // TypeError в строгом режиме
const bound = task.print.bind(task);
bound();
```

Для обычной функции this зависит от вызова.

В task.print() это task.

bind закрепляет this.

Стрелка берёт this из внешнего окружения

Класс, конструктор и экземпляр

```
class Task {
  constructor(title) {
    this.title = title;
    this.done = false;
  }
  complete() { this.done = true; }
}

const task = new Task("JS");
task.complete();
console.log(task.done);
```

new создаёт экземпляр.

constructor задаёт начальное состояние.

Метод complete находится в прототипе класса.

Объекты в JavaScript используют прототипную модель

Модули: код в нескольких файлах

```
// tasks.js
export function isOpen(task) {
  return !task.done;
}

// main.js
import { isOpen } from "./tasks.js";
console.log(isOpen({ done: false }));
```

export открывает имя другим модулям.

import подключает его.

В браузере подключаем main.js через type="module".

У каждого модуля своя область видимости

Ошибки и try...catch

```
function parseCount(text) {
  const n = Number(text);
  if (!Number.isFinite(n)) {
    throw new Error("Нужно число");
  }
  return n;
}
try {
  console.log(parseCount("кот"));
} catch (error) {
  console.log(error.message);
}
```

throw прерывает обычный путь выполнения.

catch получает ошибку.

finally нужен для завершающих действий.

Обрабатываем ошибку там, где можем осмысленно на неё отреагировать

JavaScript меняет страницу

```
// В HTML есть <h1 id="title">Задачи</h1>
const title = document.querySelector("#title");

if (title !== null) {
  title.textContent = "Мои задачи";
}
```

querySelector ищет элемент по CSS-селектору.

textContent задаёт текст.

Без подходящего элемента получим null.

DOM API предоставляет браузер

Событие и callback

```
// В HTML есть <button id="add">Добавить</button>
const button = document.querySelector("#add");

function handleClick() {
  console.log("Нажали кнопку");
}

button.addEventListener("click", handleClick);
```

Передаём функцию handleClick.

Браузер вызовет её при событии.

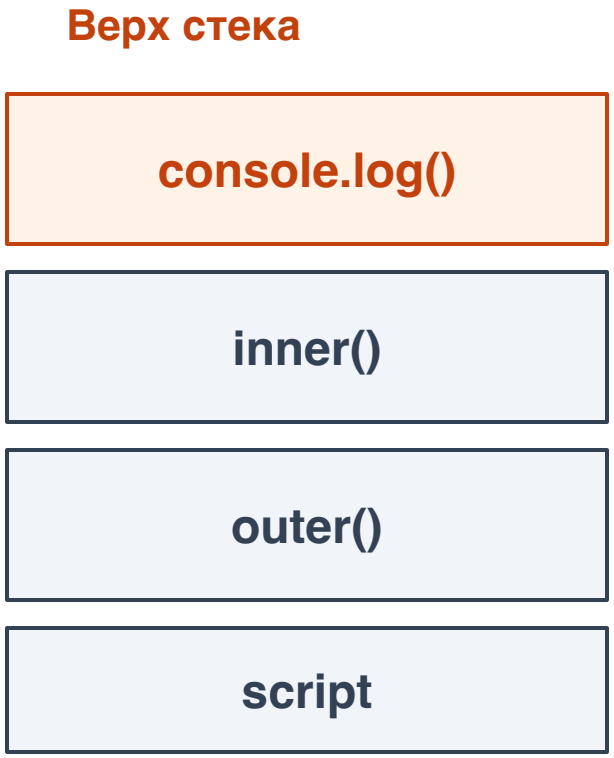
handleClick() вызвало бы её сразу.

Callback описывает, что сделать, когда другой код его вызовет

Стек вызовов

```
function outer() { inner(); }
function inner() { console.log("JS"); }
outer();
```

Момент внутри console.log



Последний начатый вызов завершается первым

Зачем нужна асинхронность

- Ответ по сети может прийти значительно позже
- Пользователь должен продолжать работать со страницей
- Программа запускает операцию и возвращает управление среде
- Когда результат готов, среда даёт выполнить продолжение

Ожидание операции не требует крутить цикл в JavaScript

Таймер: выполнить позже

```
console.log("Начало");

setTimeout(() => {
  console.log("Таймер");
}, 0);

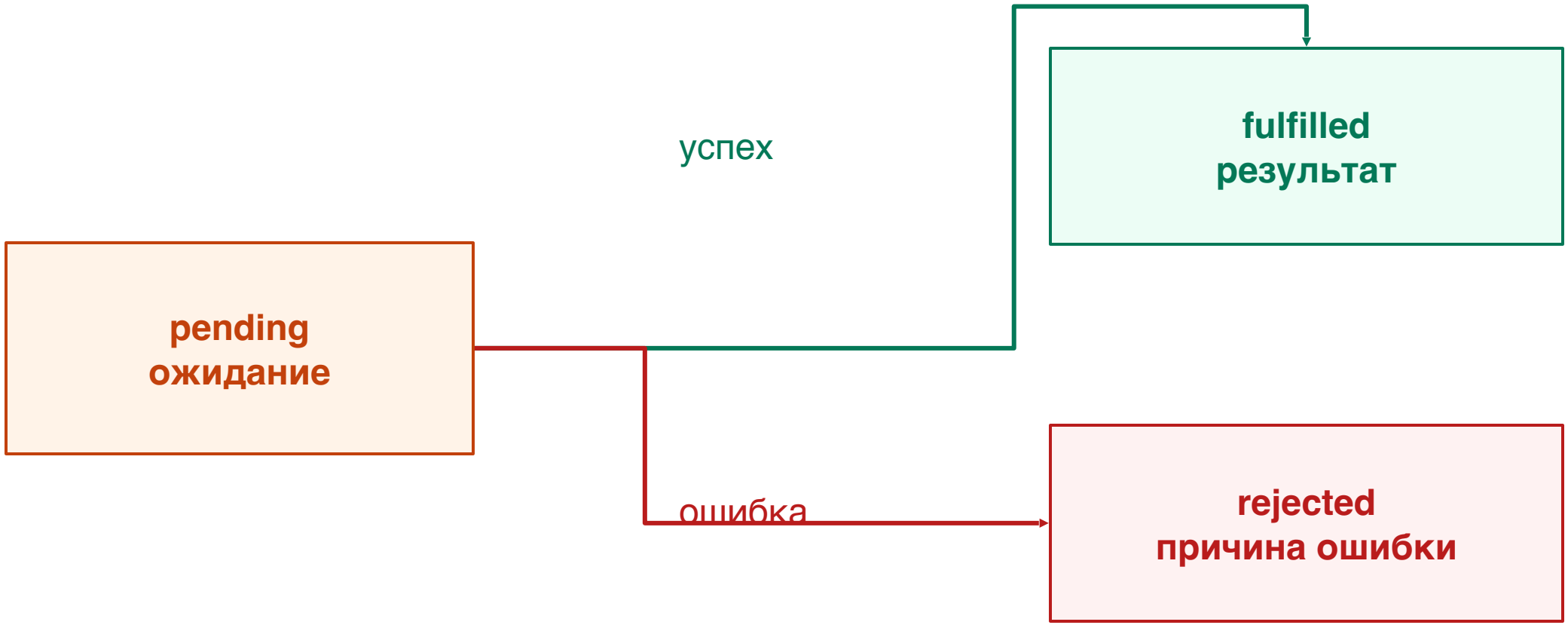
console.log("Конец");
```

Вывод:
Начало
Конец
Таймер

0 не означает «прямо сейчас».

Задержка задаёт порог ожидания, а не точное время запуска

Promise: будущий результат



После fulfilled или rejected состояние больше не меняется

then выполняется позже, executor сразу

```
console.log("A");
const result = new Promise(resolve => {
  console.log("B");
  resolve(42);
});
result.then(value => console.log(value));
console.log("C");
```

Вывод:
A
B
C
42

Тело, переданное в new Promise, выполняется синхронно.

Обработчик then не запускается прямо внутри resolve

Цепочка Promise и ошибки

```
Promise.resolve("  JS  ")
  .then(title => title.trim())
  .then(title => {
    console.log(title);
    throw new Error("Не удалось сохранить");
  })
  .catch(error => console.log(error.message))
  .finally(() => console.log("Завершено"));
```

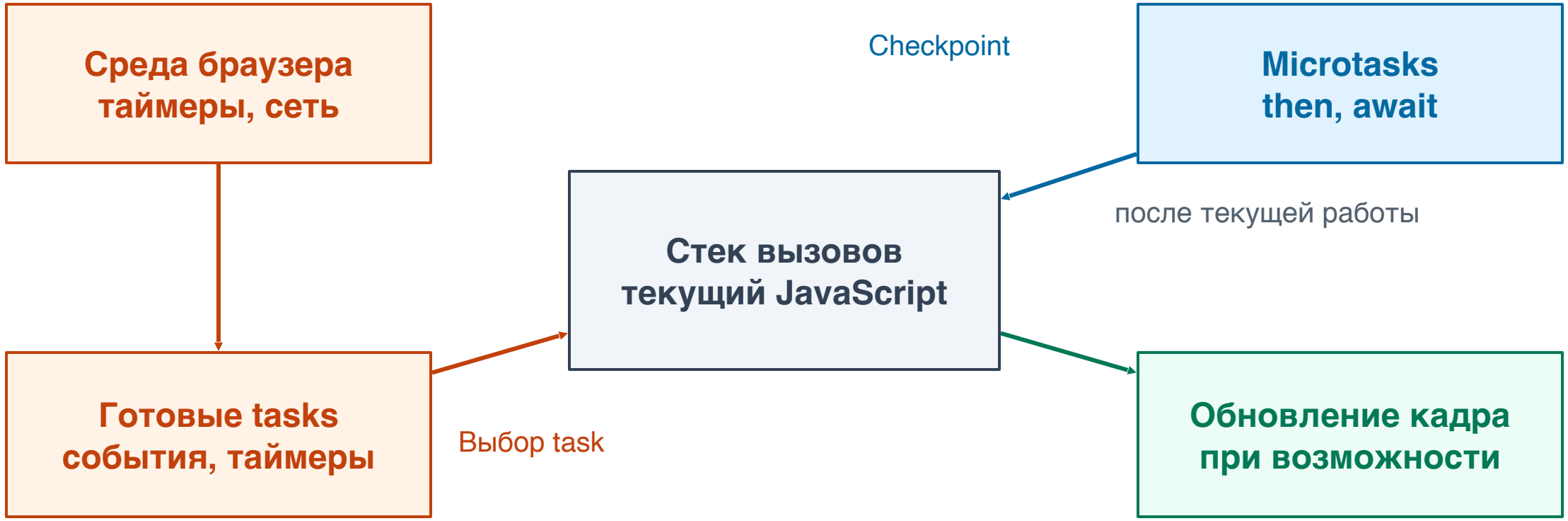
Результат then переходит следующему обработчику.

Выброшенная ошибка отклоняет следующий Promise.

catch может восстановить успешную цепочку.

Из then нужно вернуть результат или следующий Promise

Event loop в браузере



Сначала завершается текущая работа, затем выполняются готовые продолжения

В каком порядке появятся буквы?

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Есть обычный код, таймер и обработчик Promise.

Запишите порядок до следующего слайда.

Рассматриваем один запуск скрипта в браузере

Шаг 1. Выполняется обычный код

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Стек вызовов

script

Таймер в среде

ещё не создан

Microtasks

пусто

Tasks нашего примера

пусто

Вывод: A

console.log("A") выполняется сразу.

Шаг 2. Таймер зарегистрирован

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Стек вызовов

script

Таймер в среде

В ждёт допуска после задержки

Microtasks

пусто

Tasks нашего примера

пусто

Вывод: A

setTimeout передаёт таймер среде и возвращает управление.

Шаг 3. Микрозадача и конец скрипта

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Стек вызовов

пусто

Таймер в среде

В ждёт допуска
или уже готов

Microtasks

C

Tasks нашего примера

пусто

Вывод: A D

then поставил C в микрозадачи. Скрипт напечатал D и завершился.

Шаг 4. Выполняется микрозадача

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Стек вызовов

callback C

Таймер в среде

В ждёт допуска
или уже готов

Microtasks

пусто

Tasks нашего примера

пусто

Вывод: A D C

До выбора следующей task среда выполняет микрозадачи.

Шаг 5. Таймер готов к запуску

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Стек вызовов

пусто

Таймер в среде

задержка прошла

Microtasks

пусто

Tasks нашего примера

B

Вывод: A D C

Микрозадач нет. Готовый B находится среди задач для выбора.

Шаг 6. Выполняется задача таймера

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Стек вызовов

callback B

Таймер в среде

задержка прошла

Microtasks

пусто

Tasks нашего примера

пусто

Вывод: A D C B

Когда таймер допущен к запуску, event loop выбирает его task.

В основе действительно цикл

```
// Псевдокод, а не код для страницы
while (isRunning) {
  const task = takeRunnableTask();
  if (task) run(task);

  drainMicrotasks();

  if (hasRenderingOpportunity()) {
    updateRendering();
  }
  waitForWorkIfIdle();
}
```

Среда выбирает работу и повторяет шаги.

При отсутствии работы может ждать события ОС.

Такой while в своём скрипте заблокирует страницу.

Упрощено: отрисовка не обязательна на каждом обороте

Настоящий исходник: цикл libuv

```
while (r != 0 && loop->stop_flag == 0) {
  /* ... */
  uv__run_pending(loop);
  uv__run_idle(loop);
  uv__run_prepare(loop);
  /* ... */
  uv__io_poll(loop, timeout);
  /* ... */
  uv__run_check(loop);
  uv__run_closing_handles(loop);
  /* ... */
}
```

C, libuv v1.51.0
src/unix/core.c
Функция uv_run

Фрагмент сокращён.

Node.js использует libuv.

Это цикл ввода-вывода Node.js, не реализация браузерных microtasks

Браузер и Node.js: общая идея, разные детали

Браузер	Node.js
События страницы и сеть	Сеть, файлы, другие системные операции
Возможности обновления кадра	Браузерной отрисовки нет
Правила HTML event loop	Фазы libuv и правила Node.js
Promise и microtasks	Promise, microtasks и дополнительные механизмы

Порядок из браузера нельзя без проверки переносить на все API Node.js

Микрозадачи могут добавлять микрозадачи

```
queueMicrotask(() => {
  console.log("M1");
  queueMicrotask(() => console.log("M2"));
});
setTimeout(() => console.log("T"), 0);
console.log("S");
```

Вывод:
S
M1
M2
T

Checkpoint продолжает опустошать очередь.

Бесконечная цепочка микрозадач может задержать tasks и кадр

async и await

```
async function getCount() {
  const count = await Promise.resolve(3);
  return count + 1;
}

getCount().then(value => console.log(value));
console.log("Дальше");
```

async-функция возвращает Promise.

await приостанавливает её продолжение.

Вывод: Дальше, 4.

Остальной код может выполняться, пока эта функция ожидает

Что происходит до и после await

```
async function run() {
  console.log("1");
  await Promise.resolve();
  console.log("2");
}

console.log("A");
run();
console.log("B");
```

Вывод:
A
1
B
2

До первого await функция выполняется обычным образом.

Продолжение после await выполняется асинхронно

Запрос данных через fetch

```
async function loadTasks() {
  const response = await fetch("/api/tasks");
  if (!response.ok) {
    throw new Error("HTTP " + response.status);
  }
  return await response.json();
}
```

fetch возвращает Promise с Response.

json() асинхронно читает и разбирает тело.

HTTP 404 сам по себе не отклоняет fetch.

Обработка HTTP-статуса нужна отдельно от сетевой ошибки

Загрузка, ошибка и finally

```
async function refresh() {
  button.disabled = true;
  try {
    const tasks = await loadTasks();
    renderTasks(tasks);
  } catch (error) {
    message.textContent = "Ошибка загрузки";
    console.error(error);
  } finally {
    button.disabled = false;
  }
}
```

Продолжение предыдущего примера.

button и message — DOM-элементы.

renderTasks обновляет список.

finally вернёт кнопку в рабочее состояние при успехе и ошибке

Независимые операции: Promise.all

```
const tasksPromise = loadTasks();
const profilePromise = loadProfile();

const [tasks, profile] = await Promise.all([
  tasksPromise,
  profilePromise
]);
```

Пример внутри async-функции или модуля.

Обе операции уже запущены.

Результаты сохраняют порядок входного массива.

Promise.all отклонится при первой ошибке, но не отменит остальные операции

async не переносит вычисления в другой поток

```
async function calculate() {
  let sum = 0;
  for (let i = 0; i < 1_000_000_000; i++) {
    sum += i;
  }
  return sum;
}

// Вызов займёт основной поток надолго
```

Цикл выполняется синхронно.

Слово async меняет форму результата, но не место вычисления.

await Promise.resolve() тоже не гарантирует возможность отрисовать кадр

Тяжёлые вычисления и Web Worker

- Worker выполняет код в отдельном контексте
- Основной поток и Worker обмениваются сообщениями
- Прямого доступа к DOM у Worker нет
- Для небольших порций работы можно возвращать управление среде

Параллельное вычисление и асинхронное ожидание — разные задачи

Мини-задача: порядок выполнения

```
console.log("1");
setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => {
  console.log("3");
  queueMicrotask(() => console.log("4"));
});

console.log("5");
```

Запишите порядок.

Для каждой строки укажите: обычный код, microtask или task.

Сначала ответ, затем объяснение через очереди

Разбор: 1, 5, 3, 4, 2

1 и 5	Текущий скрипт
3, затем 4	Микрозадачи до опустошения очереди
2	Готовая задача таймера

Микрозадача 4 добавилась во время того же checkpoint

Практика: преобразование списка задач

```
const tasks = [
  { id: 1, title: "Git", done: true },
  { id: 2, title: "JS", done: false },
  { id: 3, title: "CSS", done: false }
];

// Написать openTitles(tasks)
// Ожидается: ["JS", "CSS"]
```

Вернуть названия незавершённых задач.

Не изменять исходный массив.

Проверить пустой список и список выполненных задач.

Сначала решение с циклом, затем вариант с filter и map

Практика: ожидание без блокировки

```
function delay(ms) {
  return new Promise(resolve => {
    setTimeout(resolve, ms);
  });
}

// Написать async-функцию showLater()
// Сначала вывести "Загрузка"
// Через delay(500) вывести "Готово"
```

Использовать await.

Сразу после вызова showLater вывести "Дальше".

Предсказать полный порядок.

Ожидается: Загрузка, Дальше, Готово

Демонстрация после лекции

- Подключение main.js и работа с Console
- Список задач: функции, filter, map и DOM
- Событие кнопки и состояние загрузки
- Порядок выполнения через breakpoints и console.log
- Успех и ошибка запроса в Network

Один небольшой проект, который связывает изученные части

Что важно унести с собой

- Функция — значение, объект может быть общим для нескольких переменных
- const запрещает переприсваивание, но не изменение объекта
- Обработчик выполняется до конца, другие задачи ждут
- Promise и await задают продолжения через асинхронный механизм
- Долгие вычисления на основном потоке мешают интерфейсу

Следующий шаг: небольшой интерфейс на чистом JavaScript

Источники и дальнейшее чтение

- MDN JavaScript Guide: язык, функции, объекты и модули
- MDN: Promise, async / await и microtasks
- WHATWG HTML: алгоритм браузерного event loop
- Node.js Learn: event loop и таймеры
- libuv v1.51.0: uv_run в src/unix/core.c

Ссылки на конкретные разделы находятся в заметках слайдов